

3. KV3 - Izrada prototipne vizualizacije podataka

[Razrada koncepta, definiranje funkcionalnosti i ponašanja - izrada prototipa.]

3.1. Osnovne funkcionalnosti i ponašanja

[Navesti osnovne funkcionalnosti vizualizacije i njihovo ponašanje]

Z-3.1.1. Identificirati ključne funkcionalnosti koje će biti potrebne za prikaz podataka.

Potrebno je mijenjati boju na karti svijeta prilikom prelaska, kreirati polja za ispis podataka o pojedinim državama. Potrebno je kreirati padajući izbornik u kojem će biti popis religija svijeta

Z-3.1.2. Definirati osnovne vrste ponašanja.

Prikaz podataka o državi uz zoom na odabranu državu prilikom klika na državu. Prikaz generalnih podataka o svijetu pritiskom na gumb. Prikaz podataka o kontinentu pritiskom na kontinent. Prilagodba karte svijeta u ovisnosti o tome koja je religija odabrana iz padajućeg izbornika

Z-3.1.3. Odabrati elemente s kojima će korisnici moći vršiti interakciju i definirati interakcije između korisnika i vizualizacije s pripadajućim opisom.

Elementi s kojima će korisnici vršiti interakciju: dvije karte svijeta, slider, gumb, padajući izbornik.

Interakcije s vizualizacijom: pritisni na kartu svijeta za pregled podataka, prelazak pokazivacem preko karte svijeta za pregled imena država/kontinenata

3.2. Napredne funkcionalnosti i ponašanja:

[Navesti napredne funkcionalnosti vizualizacije i njihovo ponašanje]

Z-3.2.1. Identificirati napredne funkcionalnosti koje će biti potrebne za analizu podataka.

Pretazivanje podataka po stupcima, filtriranje podataka prema vrijednosti (ne prikazivati ako je vrijednost 0), parsiranje podataka, neki podaci sadrže decimale, ili zareze točke za odvajanje tisućinki (npr: 1234 = 1,234 tisuću dvjesto trideset i četiri)

Z-3.2.2. Definirati napredne vrste ponašanja

Zoomiranje na kartu svijeta, korisnik može pobliže pogledati neku državu ako na nju zoomira

Z-3.2.3. Definirati interakcije koje će omogućiti korisnicima dodatnu analizu podataka.

Ako korisnik odabere iz padajućeg izbornika religiju kartu svijeta je potrebno obojati u ovisnosti o gustoći populacije određene religije sto zahtjeva dodatnu analizu, filtriranje i pregled podataka

3.3. Implementacija osnovnih funkcionalnosti

[Opisati i dokazima potkrijepiti proces implementacije osnovnih funkcionalnosti]

Z-3.3.1. Izraditi kod koji omogućava prethodno definirane funkcionalnosti.
Dokazati opisom primjera koda.

```
// Kod koji služi za prikaz globalnih podataka pritiskom na gumb
// button.js
import { columnMapping, formatNumber } from './utils.js';
let data = [];
try {
  data = await d3.csv("data/WRP_global.csv");
} catch (error) {
  console.error("Error loading CSV:", error);
}

document.getElementById("year-slider").addEventListener("input", () => {
  document.getElementById("selected-year").textContent = `Year:
${document.getElementById("year-slider").value}`;
});

document.getElementById("show-data-btn").addEventListener("click", (event) => {
  event.stopPropagation();
  const year = document.getElementById("year-slider").value;
  const popup = d3.select("#popup");
  const container = d3.select(".container");

  const row = data.find(d => +d["year"] === +year);
  let html = '<button class="close-btn">X</button>';
  if (!row) {
    html += `<strong>No data available for ${year}</strong>`;
  } else {
    const entries = Object.entries(row).filter(([key]) => key !== "year" && key !== "version");
    html += `<strong>Data for Year ${year}</strong>
<table>
  <tr>
    <th>Metric</th>
    <th>Value</th>
  </tr>`;
    entries.forEach(([key, value]) => {
      const fullName = columnMapping[key] || key;
      const displayName = fullName.includes("Total") ? `<strong>${fullName}</strong>` : fullName;
      const formattedValue = fullName.includes("Total") ?
`<strong>${formatNumber(value)}</strong>` : formatNumber(value);
      html += `<tr>
        <td>${displayName}</td>`
    });
  }
}
```

```

        <td>${formattedValue}</td>
    </tr>`;
});
html += `</table>`;
}

```

```

popup.html(html)
.style("display", "block");
container.classed("blurred", true);

```

```

d3.select("#popup .close-btn").on("click", () => {
    popup.style("display", "none");
    container.classed("blurred", false);
});
});

```

Index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <title>Interactive World Map</title>
    <link rel="stylesheet" href="css/style.css" />
    <link rel="stylesheet" href="css/toggle_switch.css" />
    <link rel="stylesheet" href="css/reset_button.css" />
    <link rel="stylesheet" href="css/year_slider.css" />
    <link rel="stylesheet" href="css/show_data_button.css" />
    <link rel="stylesheet" href="css/popup_global_data.css" />
    <link rel="stylesheet" href="css/infobox.css" />
    <script src="https://d3js.org/d3.v7.min.js"></script>
</head>
<body>
    <div class="container">
        <div id="map-container">
            <div id="info-box">Select a country</div>
            <svg id="map"></svg>
        </div>
        <div id="sidebar">
            <div id="controls">
                <h2 id="selected-year">Year: 1945</h2>
                <input type="range" id="year-slider" min="1945" max="2010" step="5" value="1945" />
                <br /><br />
                <button id="reset-btn" style="display:none;">Back to the World view</button>
                <div id="view-toggle">

```

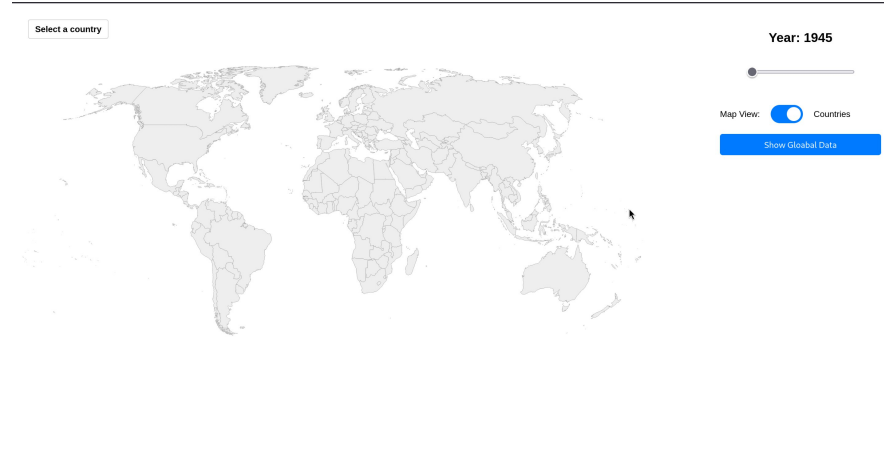
```

<label for="map-view-switch">Map View: </label>
<label class="switch">
  <input type="checkbox" id="map-view-switch" checked>
  <span class="slider round"></span>
</label>
<span id="view-label">Countries</span>
</div>
</div>
<button id="show-data-btn">Show Gloabal Data</button>
</div>
</div>
<div id="popup"></div>

<script type="module" src="js/main.js"></script>
<script type="module" src="js/button.js"></script>
<script type="module" src="js/regionalData.js"></script>
<script type="module" src="js/nationalData.js"></script>
</body>
</html>

```

Z-3.3.2. Testirati funkcionalnosti i osigurati da su ispravne, tj. da funkcioniraju na očekivani način. Potrebno potvrditi slikom.





3.4. Implementacija osnovnog ponašanja

[Opisati i dokazima potkrijepiti proces implementacije osnovnog ponašanja]

Z-3.4.1. Izraditi kod koji omogućava ranije definirano ponašanje. Dokazati opisom primjera koda.

Kod koji služi za uzimanje podataka o tome koju kartu želimo vidjeti, prikazuje tu kartu, implementirano je zoomiranje kada se pritisne na kartu te prikaz države ili kontinenta na kojem se nalazi pokazivac preko info-boxa. Poziva funkcije koje služe za prikaz podataka koje također učitavaju iz csv datoteka

```
export async function initializeMap(g, svg, width, height, zoom, resetBtn, viewState) {  
  let selectedContinent = null;  
  let selectedCountry = null;  
  
  const geojsonFile = viewState === "continents" ? "data/world_map_continents.geojson" :  
    "data/world_map.geo.json";  
  let data;  
  try {  
    console.log("Loading GeoJSON file:", geojsonFile);  
    data = await d3.json(geojsonFile);  
    console.log("GeoJSON loaded:", data);  
  } catch (error) {  
    console.error("Failed to load GeoJSON:", error);  
    throw error;  
  }  
  
  const infoBox = d3.select("#info-box");  
  const year = d3.select("#year-slider").node().value;
```

```

const filteredFeatures = viewState === "continents"
  ? data.features.filter(f=> f.properties.CONTINENT !== "Antarctica")
  : data.features;

console.log("Filtered features:", filteredFeatures);

const projection = d3.geoNaturalEarth1();
projection.fitSize([width, height], { type: "FeatureCollection", features: filteredFeatures });
const path = d3.geoPath().projection(projection);

console.log("Rendering paths...");
const paths = g.selectAll("path")
  .data(filteredFeatures)
  .join("path")
  .attr("class", viewState === "continents" ? "continent" : "country")
  .attr("d", path)
  .on("mouseover", function (event, d) {
    d3.select(this).attr("fill", "orange");
    if (!selectedContinent && !selectedCountry) {
      const name = d.properties.name || d.properties.CONTINENT || "Unknown";
      infoBox.text(name);
    }
  })
  .on("mouseout", function (event, d) {
    d3.select(this).attr("fill", "#eee");
    if (!selectedContinent && !selectedCountry) {
      infoBox.text(viewState === "continents" ? "Select a continent" : "Select a country");
    }
  })
  .on("click", function (event, d) {
    console.log(d.properties);
    event.stopPropagation();
    let name = d.properties.name || d.properties.CONTINENT || "Unknown";

const regionMap = {
  "North America": "West. Hem",
  "South America": "West. Hem",
  "Asia": ["Asia", "Mideast"],
  "Australia": null,

```

```

    "Oceania": null,
  };
  const regions = regionMap[name] || name;

  if (viewState === "continents") {
    selectedContinent = name;
    selectedCountry = null;
    if (regions) {
      displayRegionalData(regions, name);
    } else {
      infoBox.text(`No data available for ${name}`);
    }
    resetBtn.style("display", "none");
  } else {
    selectedContinent = null;
    selectedCountry = name;
    const countryCode = getCountryCode(name, year);
    const displayName = getDisplayName(name, year);
    if (countryCode) {
      displayNationalData(countryCode, displayName);
    } else {
      infoBox.text(`No data available for ${displayName}`);
    }
  }

  const [[x0, y0], [x1, y1]] = path.bounds(d);
  const dx = x1 - x0;
  const dy = y1 - y0;
  const x = (x0 + x1) / 2;
  const y = (y0 + y1) / 2;
  const scale = Math.max(1, Math.min(8, 0.9 / Math.max(dx / width, dy / height)));
  const translate = [width / 2 - scale * x, height / 2 - scale * y];

  svg.transition()
    .duration(750)
    .call(
      zoom.transform,
      d3.zoomIdentity.translate(translate[0], translate[1]).scale(scale)
    );

  resetBtn.style("display", "inline-block");
}

```

```

});

console.log("Paths rendered:", paths.size(), "elements");

svg.on("click", function (event) {
  if (!d3.select(event.target).classed("continent") && !d3.select(event.target).classed("country")) {
    selectedContinent = null;
    selectedCountry = null;
    infoBox.text(viewState === "continents" ? "Select a continent" : "Select a country");
  }
});
}

import { displayRegionalData } from './regionalData.js';
import { displayNationalData } from './nationalData.js';
import { historicalCountryMap, getCountryCode, getDisplayName } from './utils.js';

```

Z-3.4.2. Testirati ponašanje i osigurati da je ispravno, tj. da funkcioniра na očekivani način. Potrebno potvrditi slikom.



Select a continent



Year: 1945



Map View: ☐ Continents

Show Global Data

Europe



Year: 1945



Map View: ☐ Continents

Show Global Data

Europe

Metric	Value
Christianity Protestants	83 145 512
Christianity Roman Catholics	234 941 422
Christianity Eastern Orthodox	96 503 645
Christianity Anglican	29 855 595
Christianity Others	5 361 826
Christianity Total	420 908 095
Judaism Others	6 068 020
Judaism Total	6 068 020
Islam Other	29 353 397
Islam Total	29 353 397
Buddhism Other	725 632
Buddhism Total	725 632



Year: 1945



Map View: ☐ Continents

Show Global Data